

Repository Evaluator - Technical Overview

A comprehensive tool for analyzing GitHub/Bitbucket repositories to assess their suitability for **SWE-Bench++ sample creation**. The evaluator combines repository-level metrics with detailed PR analysis and F2P/P2P (Fail-to-Pass/Pass-to-Pass) test verification.

Table of Contents

1. [Quick Start](#)
 2. [Repository-Level Analysis](#)
 3. [PR-Level Analysis & Filters](#)
 4. [F2P/P2P Test Analysis](#)
 5. [Supported Test Frameworks](#)
 6. [Validation Rules](#)
 7. [Output Format](#)
-

Quick Start

```
Shell
# GitHub repository
python repo_evaluator.py owner/repo-name --token $GITHUB_TOKEN

# Bitbucket repository
python repo_evaluator.py bitbucket:owner/repo --token $BITBUCKET_TOKEN
--platform bitbucket
```

Repository-Level Analysis

The evaluator first performs a comprehensive analysis of the repository structure:

Metrics Collected

Metric	Description
Total Files	Count of all files in repository
Source Files	Code files (excluding tests, configs, assets)
Test Files	Files matching test patterns for detected language
Test Coverage Ratio	$\text{test_files} / \text{source_files} \times 100\%$
Lines of Code (LoC)	Total, source, and test LoC
CI/CD Presence	Detection of GitHub Actions, CircleCI, Travis, etc.
Test Frameworks	Auto-detected test frameworks
Commit Activity	Total commits, recent (12mo), commit spread
Median Commit Interval	Time between commits (last 90 days)

Scoring System

Repositories are scored 0-100 based on:

- Test coverage ratio (higher = better)
- Commit activity and recency
- CI/CD presence
- Code quality indicators

PR-Level Analysis & Filters

Each merged PR goes through multiple validation filters before being accepted for F2P analysis.

PR Filters (Applied in Order)

Filter	Rejection Code	Description
Bot Detection	<code>bot_pr</code>	Rejects PRs from bot accounts (e.g., dependabot, renovate)

Filter	Rejection Code	Description
Language Check	<code>content_not_in_english</code>	PR title and body must be in English
Issue Validation	<code>issue_is_a_pr</code>	Linked issue must not be a PR itself
Issue State	<code>issue_is_not_closed</code>	Linked issue must be closed
Issue Word Count	<code>issue_word_count</code>	Issue body must have 20-2000 words
Min Test Files	<code>fewer_than_min_test_files</code>	PR must modify ≥ 1 test file
Max Non-Test Files	<code>more_than_max_non_test_files</code>	PR must modify ≤ 100 non-test files
Difficulty Check	<code>difficulty_not_hard</code>	PR must change > 5 files total
Max Test Files	<code>too_many_test_files</code>	PR must modify ≤ 15 test files
Max Changed Files	<code>too_many_changed_files</code>	PR must change ≤ 50 code files (excl. data files)
Patch Retrieval	<code>full_patch_retrieval</code>	Must be able to retrieve diff
Rust Embedded Tests	<code>rust_embedded_tests</code>	Rust files must not contain <code>#[test]</code> in source
Code Changes	<code>insufficient_code_changes</code>	Must have meaningful code changes

Data File Extensions (Excluded from counts)

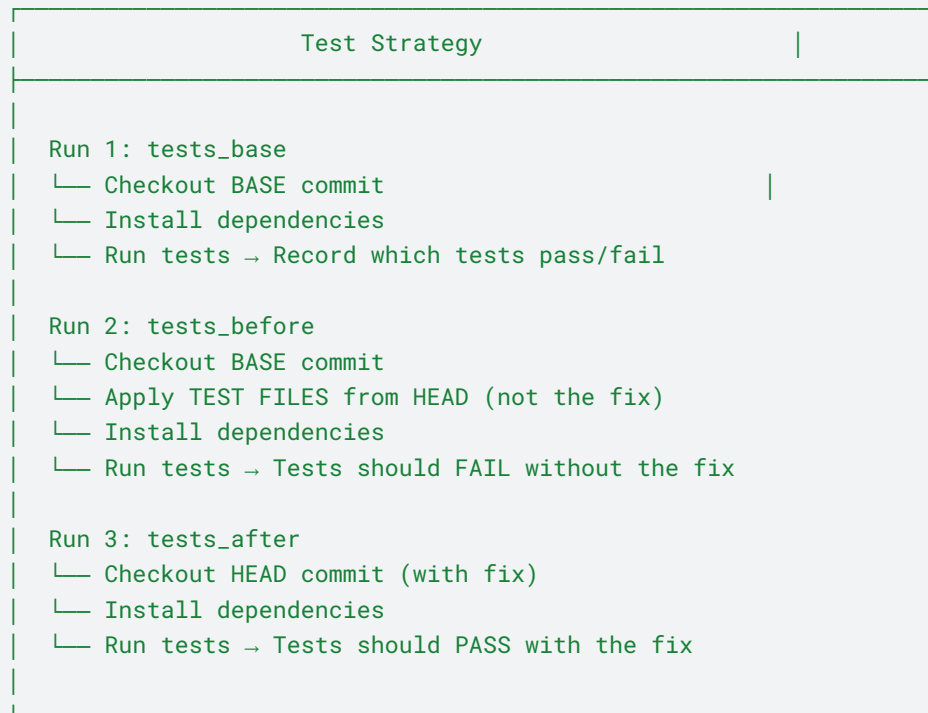
`.png, .jpg, .jpeg, .gif, .svg, .ico, .csv, .json, .xml, .yaml, .yml, .md, .txt, .pdf, .zip, .tar, .gz`

F2P/P2P Test Analysis

For accepted PRs, the evaluator performs **SWE-Bench++ compatible** test analysis using a 3-run approach. Can be skipped using flag `--skip-f2p`

The 3-Run Approach

None



Test Classification

Classification	Symbol	Meaning
FAIL_TO_PASS (F2P)	✗→✓	Test failed at base, passes at HEAD. Verifies the fix works.
PASS_TO_PASS (P2P)	✓→✓	Test passed at base, still passes at HEAD. Regression test.
FAIL_TO_FAIL (F2F)	✗→✗	Test failed at base, still fails at HEAD.

Classification	Symbol	Meaning
PASS_TO_FAIL (P2F)	✓→✗	Test passed at base, now fails. Regression introduced.

What Makes a Valid Sample

A PR is considered a **valid SWE-Bench sample** if it has:

- **At least 1 F2P test** (proves the PR fixes something)
- **At least 1 P2P test** (proves existing tests still pass - no regressions)
- No P2F tests (no regressions introduced)
- All validation rules pass (see below)

Note: PRs that add entirely new features with no pre-existing tests will be rejected because they have no P2P tests. This ensures the PR is a bug fix or feature enhancement to existing, tested code.

Supported Test Frameworks

The evaluator auto-detects and runs tests using language-specific runners:

JavaScript/TypeScript

Framework	Detection	Command
Jest	<code>jest</code> in deps, <code>jest.config.*</code>	<code>npm test --</code> or <code>npm exec -- jest</code>
Vitest	<code>vitest</code> in deps, <code>vitest.config.*</code>	<code>npm exec -- vitest run</code>
Mocha	<code>mocha</code> in deps	<code>npm exec -- mocha</code>
Node:test	<code>node --test</code> in scripts (Node 18+)	<code>npm test</code>

Python

Framework	Detection	Command
Pytest	<code>pytest</code> in deps, <code>pytest.ini</code> , <code>conftest.py</code>	<code>python -m pytest</code>
Unittest	<code>test_*.py</code> files	<code>python -m unittest discover</code>

Go

Framework	Detection	Command
Go Test	<code>go.mod</code> , <code>*_test.go</code> files	<code>go test ./...</code>

Rust

Framework	Detection	Command
Cargo Test	<code>Cargo.toml</code>	<code>cargo test</code>

Java/JVM

Framework	Detection	Command
Maven	<code>pom.xml</code>	<code>mvn test</code>
Gradle	<code>build.gradle</code>	<code>gradle test</code>
sbt	<code>build.sbt</code>	<code>sbt test</code>

Ruby

Framework	Detection	Command
RSpec	<code>rspec</code> in Gemfile, <code>spec/</code> dir	<code>bundle exec rspec</code>
Minitest	<code>minitest</code> in Gemfile	<code>bundle exec rake test</code>

.NET

Framework	Detection	Command
dotnet test	*.csproj, *.sln	dotnet test

C/C++

Framework	Detection	Command
CMake	CMakeLists.txt with test config	cmake --build && ctest
Make	Makefile with test target	make test
GoogleTest	gtest in CMake or sources	cmake --build && ctest

Validation Rules

After F2P/P2P calculation, results are validated against these rules:

F2P/P2P Validation Rules

Rule	Rejection Code	Description
No F2P Tests	empty_f2p	Must have at least 1 F2P test
No P2P Tests	empty_p2p	Must have at least 1 P2P test
Unstable Test Names	unstable_test_name	Test names must not contain timestamps, UUIDs, durations (JS/TS/C++ only)
Duplicate Names	duplicate_test_names	All test names must be unique
Failed Base in P2P	failed_base_in_p2p	P2P tests must have passed at base
Failed After	failed_after_in_f2p_p2p	All F2P/P2P tests must pass at HEAD

Rule	Rejection Code	Description
P2P Not in Base	p2p_missing_base_not_passing_before	P2P tests missing from base must pass in before
Test Coverage	test_not_in_all_stages	Tests must run in all 3 stages (exceptions for new test files)

Unstable Test Name Patterns (Auto-rejected)

None

- Unix timestamps (10-13 digits): 1704067200000
- ISO dates: 2024-01-01T00:00:00
- Build times: "built in 1.73s"
- Durations: "in 500ms", "in 2 seconds"
- UUIDs: a1b2c3d4-e5f6-...
- Memory addresses: 0x7fff5fbff8c0

Output Format

Console Output

None

```
=====
REPOSITORY EVALUATION REPORT
=====
Repository: owner/repo-name
Language: TypeScript
Overall Score: 75.0/100
Recommendation:  GOOD

--- Repository Metrics ---
Total files: 500
Source files: 350
Test files: 80
Test coverage ratio: 22.9%
...

--- PR Analysis ---
```

```
Total PRs Analyzed: 50
Accepted PRs: 12
Accepted: 12 (24.0%)
Rejected: 38 (76.0%)

Rejection Breakdown:
  fewer_than_min_test_files: 15 (39.5%)
  content_not_in_english: 8 (21.1%)
  bot_pr: 5 (13.2%)
  empty_p2p: 4 (10.5%)
  ...

--- F2P Analysis Results ---
PRs with valid F2P: 8 (66.7%)
Total F2P tests found: 45
Total P2P tests found: 230
```

JSON Output (with `--output` flag)

```
JSON
{
  "repo_full_name": "owner/repo",
  "primary_language": "TypeScript",
  "overall_score": 75.0,
  "metrics": { ... },
  "pr_analysis": {
    "accepted": 12,
    "rejected": 38,
    "acceptance_rate": 24.0,
    "rejection_breakdown": { ... },
    "f2p_results": [
      {
        "pr_number": 123,
        "f2p_count": 5,
        "p2p_count": 20,
        "verdict": "VALID"
      }
    ]
  }
}
```

Runtime Version Detection

The evaluator checks if the system's runtime version is compatible with the repository's requirements:

Language	Version Sources
Node.js	<code>.nvmrc</code> , <code>.node-version</code> , <code>package.json</code> engines
Python	<code>Dockerfile</code> , <code>pyproject.toml</code> , <code>setup.py</code> , <code>setup.cfg</code>
Go	<code>go.mod</code>
Rust	<code>rust-toolchain.toml</code> , <code>Cargo.toml</code>
Ruby	<code>.ruby-version</code> , <code>Gemfile</code>
Java	<code>pom.xml</code> , <code>build.gradle</code>
.NET	<code>global.json</code> , <code>*.csproj</code>

If incompatible, the PR is rejected with `RUNTIME_VERSION_MISMATCH`.

Monorepo Support

The evaluator detects monorepos and runs tests per-package:

Package Detection Markers

- `package.json` (JavaScript)
- `requirements.txt`, `pyproject.toml`, `setup.py` (Python)
- `Gemfile` (Ruby)
- `Cargo.toml` (Rust)
- `go.mod` (Go)
- `pom.xml`, `build.gradle` (JVM)

PR-Aware Package Detection

Only packages affected by the PR's changed files are tested, avoiding unnecessary test runs in unrelated packages.

Error Codes Reference

Code	Meaning
<code>BUILD_FAILED</code>	Dependency install or build failed
<code>TIMEOUT</code>	Test execution exceeded timeout
<code>NO_TESTS</code>	No tests found/detected
<code>NO_TEST_RUNNER</code>	No compatible test runner found
<code>NO_CHANGED_FILES</code>	Could not determine changed files
<code>RUNTIME_VERSION_MISMATCH</code>	System runtime incompatible with repo requirements
<code>INSTALL_FAILED</code>	<code>npm install</code> / <code>pip install</code> failed
<code>empty_f2p</code>	No F2P tests found
<code>empty_p2p</code>	No P2P tests found

Architecture

```
None
repo_evaluator/
├─ repo_evaluator.py          # Main entry point, PR analysis
├─ repo_evaluator_helpers.py  # Utility functions, language configs
├─ test_runners/
│   ├─ base.py                # Base classes, TestResult, F2PP2PResult
│   ├─ analyzer.py            # F2P/P2P orchestrator (3-run approach)
│   ├─ registry.py            # Runner auto-detection
│   └─ parsers.py             # Test output parsers
```

```
|— javascript.py      # Jest, Vitest, Mocha runners
|— python.py         # Pytest, Unittest runners
|— go.py             # Go test runner
|— rust.py           # Cargo test runner
|— jvm.py            # Maven, Gradle, sbt runners
|— ruby.py           # RSpec, Minitest runners
|— dotnet.py         # .NET test runner
|— c_cpp.py          # CMake, Make, GoogleTest runners
```

Limitations & Known Issues

1. **Shallow Clones:** By default, clones are shallow. Some PR commits may need fetching via API.
2. **Squash Merges:** PR base/head commits may not exist after squash merge.
3. **Test Infrastructure Changes:** If the test framework was added in the PR itself, base tests will fail.
4. **Private Dependencies:** Repos requiring private npm/pip packages may fail installation.
5. **External Services:** Tests requiring databases, APIs, etc. will likely fail.

Last updated: January 2026